# LAB: Input Capture - Ultrasonic

**Date:** 2024-10-17

**Author/Partner:** HWANG MooYeol

**Github:** repository link

**Demo Video:** Youtube link

**PDF version:**LAB InputCapture PDF

# Introduction

In this lab, you are required to create a simple program that uses input capture mode to measure the distance using an ultrasonic distance sensor. The sensor also needs trigger pulses that can be generated by using the timer output.

You must submit

- LAB Report (*.pdf & *.md)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
  - Only the source files. Do not submit project files

## Requirement

### Hardware

- MCU
  - NUCLEO-F411RE
- Actuator/Sensor/Others:
  - HC-SR04
  - breadboard

### Software

- Keil uVision, CMSIS, EC_HAL library

---

# Problem 1: Create HAL library

## Create HAL library

Declare and Define the following functions in your library.

- ecCAP2_student.h, ecCAP2_student.c

**ecCAP2.h**

```
/* Input Capture*/
// ICn selection according to CHn
#define FIRST 1
```

```c
#define SECOND 2

// Edge Type
#define IC_RISE 0
#define IC_FALL 1
#define IC_BOTH 2

// IC Number
#define IC_1 1
#define IC_2 2
#define IC_3 3
#define IC_4 4

void ICAP_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN);
void ICAP_init(PinName_t pinName);
void ICAP_setup(PinName_t pinName, int ICn, int edge_type);
void ICAP_counter_us(PinName_t pinName, int usec);
uint32_t ICAP_capture(TIM_TypeDef* TIMx, uint32_t ICn);
```

**ecCAP2.c**

```c
#include "ecICAP2.h"
#include "ecGPIO2_student.h"
#include "ecTIM2.h"
#include <math.h>
/* -------- Timer Input Capture -------- */

void ICAP_init(PinName_t pinName){
// 0. Match Input Capture Port and Pin for TIMx
   GPIO_TypeDef *port;
   unsigned int pin;
   ecPinmap(pinName, &port, &pin);
   TIM_TypeDef *TIMx;
   int TIn;

   ICAP_pinmap(pinName, &TIMx, &TIn);
   int ICn = TIn;                                        // (default) TIx=ICx

// GPIO configuration ------------------------------------------------------------
----------
// 1. Initialize GPIO port and pin as AF
   GPIO_init(pinName, AF);                        // AF=2
   GPIO_ospeed(pinName, HIGH);                     // speed VHIGH=3

// 2. Configure GPIO AFR by Pin num.
   if(TIMx == TIM1 || TIMx == TIM2)                      port->AFR[pin >> 3] |=
0x01 << (4*(pin % 8)); // TIM1~2
   else if(TIMx == TIM3 || TIMx == TIM4 || TIMx == TIM5)  port->AFR[pin >> 3] |=
0x02 << (4*(pin % 8)); // TIM3~5
   else if(TIMx == TIM9 || TIMx == TIM10|| TIMx == TIM11) port->AFR[pin >> 3] |=
0x03 << (4*(pin % 8)); // TIM9~11



// TIMER configuration ------------------------------------------------------------
-----------
// 1. Initialize Timer Interrpt
```

```
   TIM_UI_init(TIMx, 1);                        // TIMx Interrupt initialize

// 2. Modify ARR Maxium for 1MHz
   TIMx->PSC = 84-1;                                  // Timer counter clock:
1MHz(1us)  for PLL
   TIMx->ARR = 0xFFFF;                                // Set auto reload
register to maximum (count up to 65535)

// 3. Disable Counter during configuration
   TIMx->CR1 &= ~TIM_CR1_CEN;                     // Disable Counter during
configuration



// Input Capture configuration ------------------------------------------------
-------------------
// 1. Select Timer channel(TIx) for Input Capture channel(ICx)
   // Default Setting
   TIMx->CCMR1 &=    ~TIM_CCMR1_CC1S;
   TIMx->CCMR1 &=    ~TIM_CCMR1_CC2S;
   TIMx->CCMR2 &=    ~TIM_CCMR2_CC3S;
   TIMx->CCMR2 &=    ~TIM_CCMR2_CC4S;

   TIMx->CCMR1 |=    TIM_CCMR1_CC1S_0;         //01<<0   CC1S    TI1=IC1
   TIMx->CCMR1 |=    TIM_CCMR1_CC2S_0;         //01<<8   CC2s    TI2=IC2
   TIMx->CCMR2 |=    TIM_CCMR2_CC3S_0;         //01<<0   CC3s    TI3=IC3
   TIMx->CCMR2 |=    TIM_CCMR2_CC4S_0;         //01<<8   CC4s    TI4=IC4

// 2. Filter Duration (use default)

   TIMx->CCMR1 &=    ~(15<<4);               // clear IC1F
   TIMx->CCMR1 &=    ~(15<<12);              // clear IC2F
   TIMx->CCMR2 &=    ~(15<<4);               // clear IC3F
   TIMx->CCMR2 &=    ~(15<<12);              // clear IC4F

   TIMx->CCMR1 |=    (2<<4);                 // set IC1F 0b0010
   TIMx->CCMR1 |=    (2<<12);                // set IC2F 0b0010
   TIMx->CCMR2 |=    (2<<4);                 // set IC3F 0b0010
   TIMx->CCMR2 |=    (2<<12);                // set IC4F 0b0010

// 3. IC Prescaler (use default)

// 4. Activation Edge: CCyNP/CCyP
   TIMx->CCER &= ~(0b1010<< 4 * (ICn - 1) );

// 5.   Enable CCy Capture, Capture/Compare interrupt
   TIMx->CCER |= (1 << 4 * (ICn - 1));                  // CCn(ICn) Capture
Enable

// 6.   Enable Interrupt of CC(CCyIE), Update (UIE)
   TIMx->DIER |= (1 << ICn);               // Capture/Compare Interrupt Enable
for ICn
   TIMx->DIER |= TIM_DIER_UIE;                     // Update Interrupt enable

// 7.   Enable Counter
   TIMx->CR1    |= TIM_CR1_CEN;                     // Counter enable
}
```

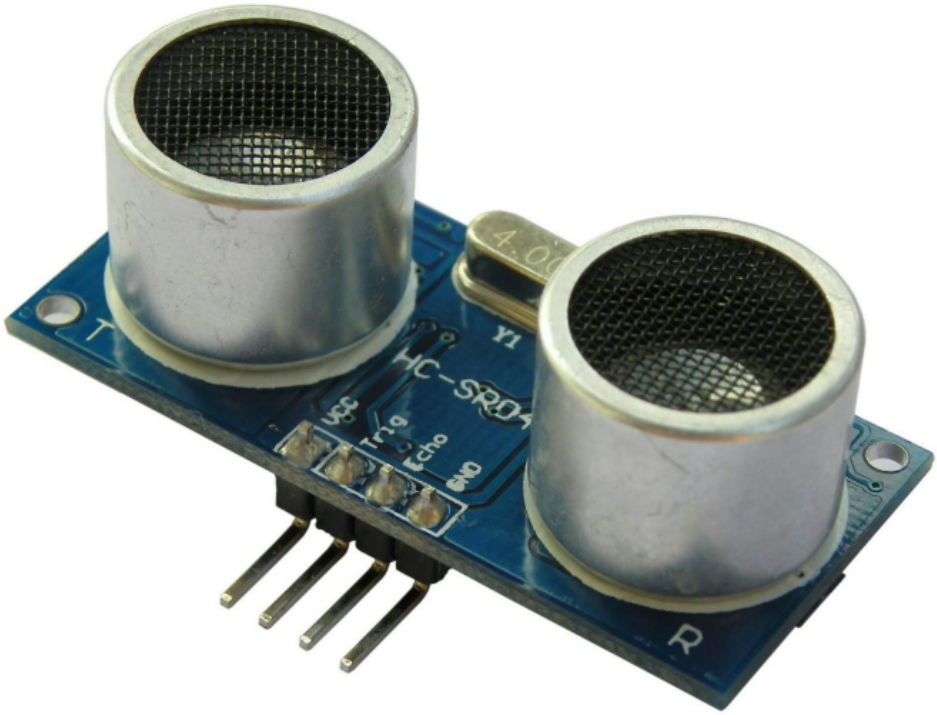
```
// Configure Selecting TIx-ICy and Edge Type
void ICAP_setup(PinName_t pinName, int ICn, int edge_type){
// 0. Match Input Capture Port and Pin for TIMx
   GPIO_TypeDef *port;
   unsigned int pin;
   ecPinmap(pinName, &port, &pin);

   TIM_TypeDef *TIMx;
   int CHn;
   ICAP_pinmap(pinName, &TIMx, &CHn);

// 1. Disable  CC. Disable CCInterrupt for ICn.
   TIMx->CCER &= ~(1 << 4 * (ICn -1));                          // Capture
Disable
   TIMx->DIER &= ~(1 << ICn);                                  // CCn Interrupt
Disable


// 2. Configure  IC number(user selected) with given IC pin(TIMx_CHn)
   switch(ICn){
         case 1:
               TIMx->CCMR1 &= ~TIM_CCMR1_CC1S;                          //reset
  CC1S
               if (ICn==CHn) TIMx->CCMR1 |= TIM_CCMR1_CC1S_0;//01<<0   CC1S
Tx_Ch1=IC1
               else TIMx->CCMR1 |=    TIM_CCMR1_CC1S_1;      //10<<0   CC1S
Tx_Ch2=IC1
               break;
         case 2:
               TIMx->CCMR1 &= ~TIM_CCMR1_CC2S;                       //reset
CC2S
               if (ICn==CHn) TIMx->CCMR1 |= TIM_CCMR1_CC2S_0;//01<<8   CC2S
Tx_Ch2=IC2
               else TIMx->CCMR1 |=    TIM_CCMR1_CC2S_1;     //10<<8   CC2S
Tx_Ch1=IC2
               break;
         case 3:
               TIMx->CCMR2 &= ~TIM_CCMR2_CC3S;                          //reset
CC3S
               if (ICn==CHn) TIMx->CCMR2 |= TIM_CCMR2_CC3S_0;//01<<0   CC3S
Tx_Ch3=IC3
               else TIMx->CCMR2 |= TIM_CCMR2_CC3S_1;         //10<<0   CC3S
Tx_Ch4=IC3
               break;
         case 4:
               TIMx->CCMR2 &= ~TIM_CCMR2_CC4S;                          //reset
  CC4S
               if (ICn==CHn) TIMx->CCMR2 |= TIM_CCMR2_CC4S_0;//01<<8   CC4S
Tx_Ch4=IC4
               else TIMx->CCMR2 |= TIM_CCMR2_CC4S_1;         //10<<8   CC4S
Tx_Ch3=IC4
               break;
         default: break;
      }


// 3. Configure Activation Edge direction
```

```
   TIMx->CCER  &= ~(0b1010 << 4*(ICn - 1));          // Clear CCnNP/CCnP bits
for ICn
   switch(edge_type){
      case IC_RISE: TIMx->CCER &= ~(0b1010 << 4 * (ICn - 1));   break; //rising:
00
      case IC_FALL: TIMx->CCER |= (0b0010 << 4 * (ICn - 1));    break;
//falling: 01
      case IC_BOTH: TIMx->CCER |= (0b1010 << 4 * (ICn - 1));    break; //both:
11
   }

// 4. Enable CC. Enable CC Interrupt.
   TIMx->CCER |= 1 << (4*(ICn - 1));                        // Capture Enable
   TIMx->DIER |= 1 << ICn;                              // CCn Interrupt enabled

}

// Time span for one counter step
void ICAP_counter_us(PinName_t pinName, int usec){
// 0. Match Input Capture Port and Pin for TIMx
   GPIO_TypeDef *port;
   unsigned int pin;
   ecPinmap(pinName, &port, &pin);
   TIM_TypeDef *TIMx;
   int CHn;
   ICAP_pinmap(pinName, &TIMx, &CHn);

// 1. Configuration Timer Prescaler and ARR
   TIMx->PSC = 84*usec-1;                   // Timer counter clock: 1us * usec
   TIMx->ARR = 0xFFFF;                  // Set auto reload register to maximum
(count up to 65535)
}

uint32_t is_CCIF(TIM_TypeDef *TIMx, uint32_t ccNum){
   return (TIMx->SR & (0x1UL << ccNum)) != 0;
}

void clear_CCIF(TIM_TypeDef *TIMx, uint32_t ccNum){
   TIMx->SR &= ~(1 << ccNum);
}

uint32_t ICAP_capture(TIM_TypeDef* TIMx, uint32_t ICn){
   uint32_t capture_Value;

   if (ICn == 1)
      capture_Value = TIMx->CCR1;
   else if (ICn == 2)
      capture_Value = TIMx->CCR2;
   else if (ICn == 3)
      capture_Value = TIMx->CCR3;
   else
      capture_Value = TIMx->CCR4;

   return capture_Value;
}
```

```c
//DO NOT MODIFY THIS
void ICAP_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN){
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);

   if(port == GPIOA) {
      switch(pin){
         case 0 : *TIMx = TIM2; *chN = 1; break;
         case 1 : *TIMx = TIM2; *chN = 2; break;
         case 5 : *TIMx = TIM2; *chN = 1; break;
         case 6 : *TIMx = TIM3; *chN = 1; break;
         //case 7: *TIMx = TIM1; *chN = 1N; break;
         case 8 : *TIMx = TIM1; *chN = 1; break;
         case 9 : *TIMx = TIM1; *chN = 2; break;
         case 10: *TIMx = TIM1; *chN = 3; break;
         case 15: *TIMx = TIM2; *chN = 1; break;
         default: break;
      }
   }
   else if(port == GPIOB) {
      switch(pin){
         //case 0: *TIMx = TIM1; *chN = 2N; break;
         //case 1: *TIMx = TIM1; *chN = 3N; break;
         case 3 : *TIMx = TIM2; *chN = 2; break;
         case 4 : *TIMx = TIM3; *chN = 1; break;
         case 5 : *TIMx = TIM3; *chN = 2; break;
         case 6 : *TIMx = TIM4; *chN = 1; break;
         case 7 : *TIMx = TIM4; *chN = 2; break;
         case 8 : *TIMx = TIM4; *chN = 3; break;
         case 9 : *TIMx = TIM4; *chN = 3; break;
         case 10: *TIMx = TIM2; *chN = 3; break;

         default: break;
      }
   }
   else if(port == GPIOC) {
      switch(pin){
         case 6 : *TIMx = TIM3; *chN = 1; break;
         case 7 : *TIMx = TIM3; *chN = 2; break;
         case 8 : *TIMx = TIM3; *chN = 3; break;
         case 9 : *TIMx = TIM3; *chN = 4; break;

         default: break;
      }
   }
}
```

## Problem 2: Ultrasonic Distance Sensor (HC-SR04)

The HC-SR04 ultrasonic distance sensor. This economical sensor provides 2cm to 400cm of non-contact measurement functionality with a ranging accuracy that can reach up to 3mm. Each HC-SR04 module includes an ultrasonic transmitter, a receiver and a control circuit.




HC-SR04

**The HC-SR04 Ultrasonic Range Sensor Features:**

- Input Voltage: 5V
- Current Draw: 20mA (Max)
- Digital Output: 5V
- Digital Output: 0V (Low)
- Sensing Angle: 30° Cone
- Angle of Effect: 15° Cone
- Ultrasonic Frequency: 40kHz
- Range: 2cm - 400cm

## Procedure

1. Connect the HC-SR04 ultrasonic distance sensor to MCU pins(PA6 - trigger, PB6 - echo), VCC and GND

## Measurement of Distance

The program needs to

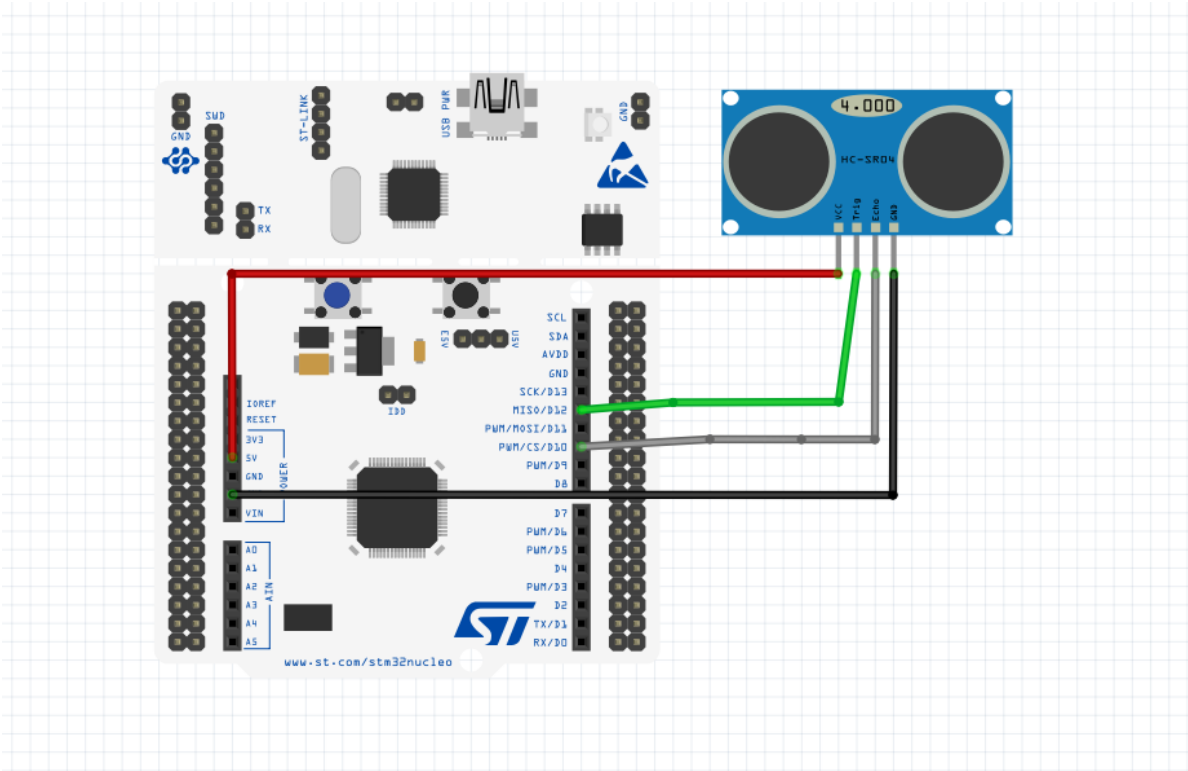
- Generate a trigger pulse as PWM to the sensor.
- Receive echo pulses from the ultrasonic sensor
- Measure the distance by calculating pulse-width of the echo pulse.
- Display measured distance in [cm] on serial monitor of Tera-Term for

  (a) 10mm (b) 50mm (c) 100mm

## Configuration

| System Clock | PWM | Input Capture |
|---|---|---|
| PLL (84MHz) | PA6 (TIM3_CH1) | PB6 (TIM4_CH1) |
| | AF, Push-Pull, No Pull-up Pull-down, Fast | AF, No Pull-up Pull-down |
| | PWM period: 50msec pulse width: 10usec | Counter Clock : 0.1MHz (10us) TI4 -> IC1 (rising edge) TI4 -> IC2 (falling edge) |

## Circuit Diagram

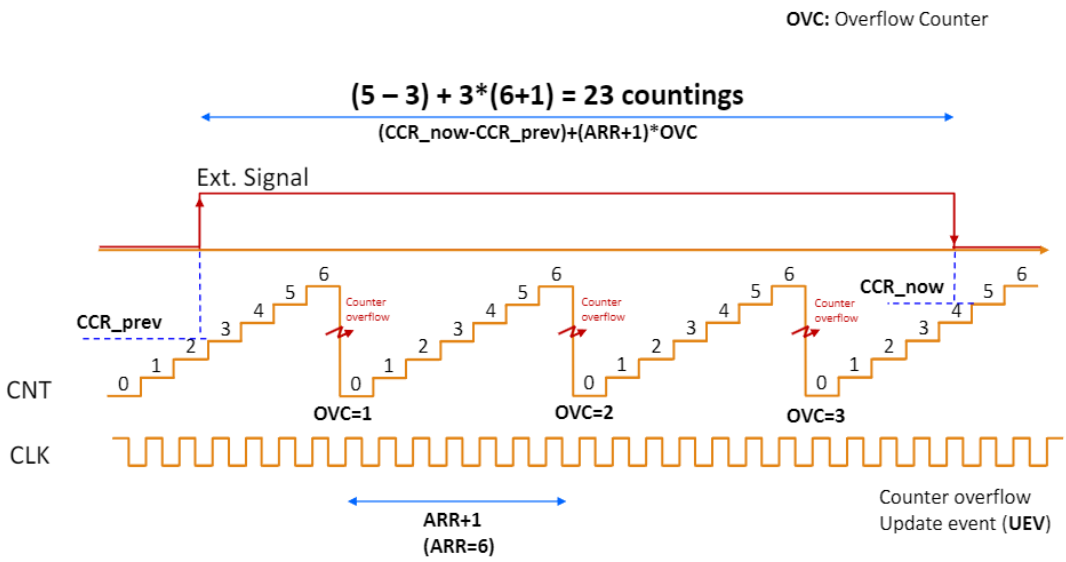



## Discussion

1. There can be an over-capture case, when a new capture interrupt occurs before reading the CCR value. When does it occur and how can you calculate the time span accurately between two captures?

Over-capture case occurs when if the time interval between the two events is shorter than the Timer Interrupt's time interval.

**time span=** Counter_period * [ (CCR_now-CCR_prev)+ OC*(ARR+1) ]




**Timer Interrupt's time interval should be adjusted longer** to calulate the time span accurately, considering the time span of the Input Capture.

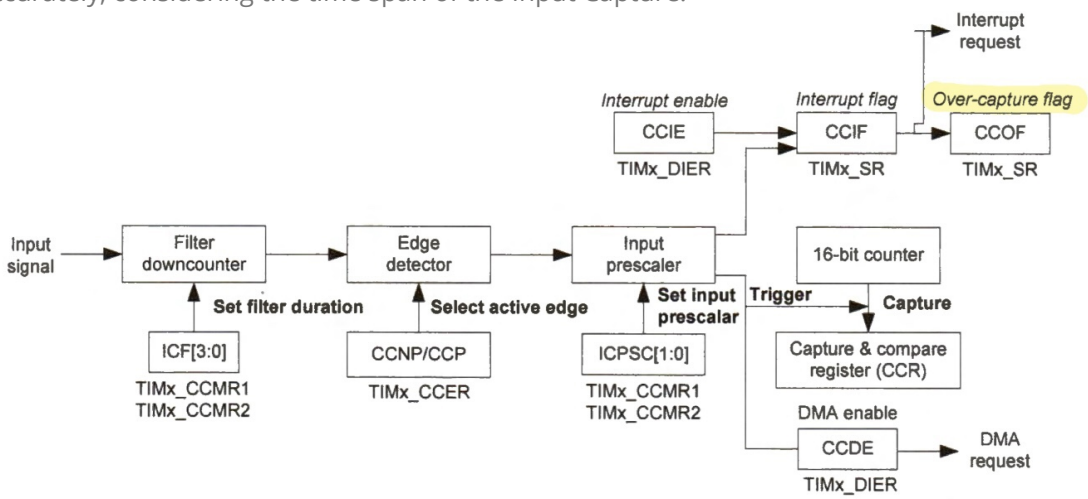
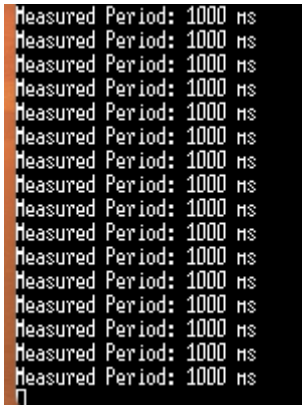



**Figure 15-21. Diagram of input capture**

2. In the tutorial, what is the accuracy when measuring the period of 1Hz square wave? Show your result.
   - Arduino code making 1[Hz] & measuring

```
const int trigPin = 10;   // Trigger pin : PWM out
const int echoPin = 7;    // Echo pin : Interrupt in

unsigned long duration;
float distance;
volatile unsigned long previousTime = 0;
volatile unsigned long currentTime = 0;
volatile unsigned long period = 0;

void setup() {
pinMode(trigPin, OUTPUT);
```

```
pinMode(echoPin, INPUT);
pinMode(2, INPUT);
attachInterrupt(digitalPinToInterrupt(2), measurePeriod, RISING);

Serial.begin(9600);
}

void loop() {
digitalWrite(trigPin, HIGH);
delayMicroseconds(500000);
digitalWrite(trigPin, LOW);
delayMicroseconds(500000);

if (period > 0) {
 Serial.print("Measured Period: ");
 Serial.print(period);
 Serial.println(" ms");
 period = 0;
}
}

void measurePeriod() {
currentTime = millis();
period = currentTime - previousTime;
previousTime = currentTime;
}
```




$$T = \frac{1}{f} = \frac{1}{1}Hz = 1sec = 1000ms$$

$$accuracy; error = \frac{1000 - 1000}{1000} \times 100 = 0$$

## Code

My code is here: [repository link]

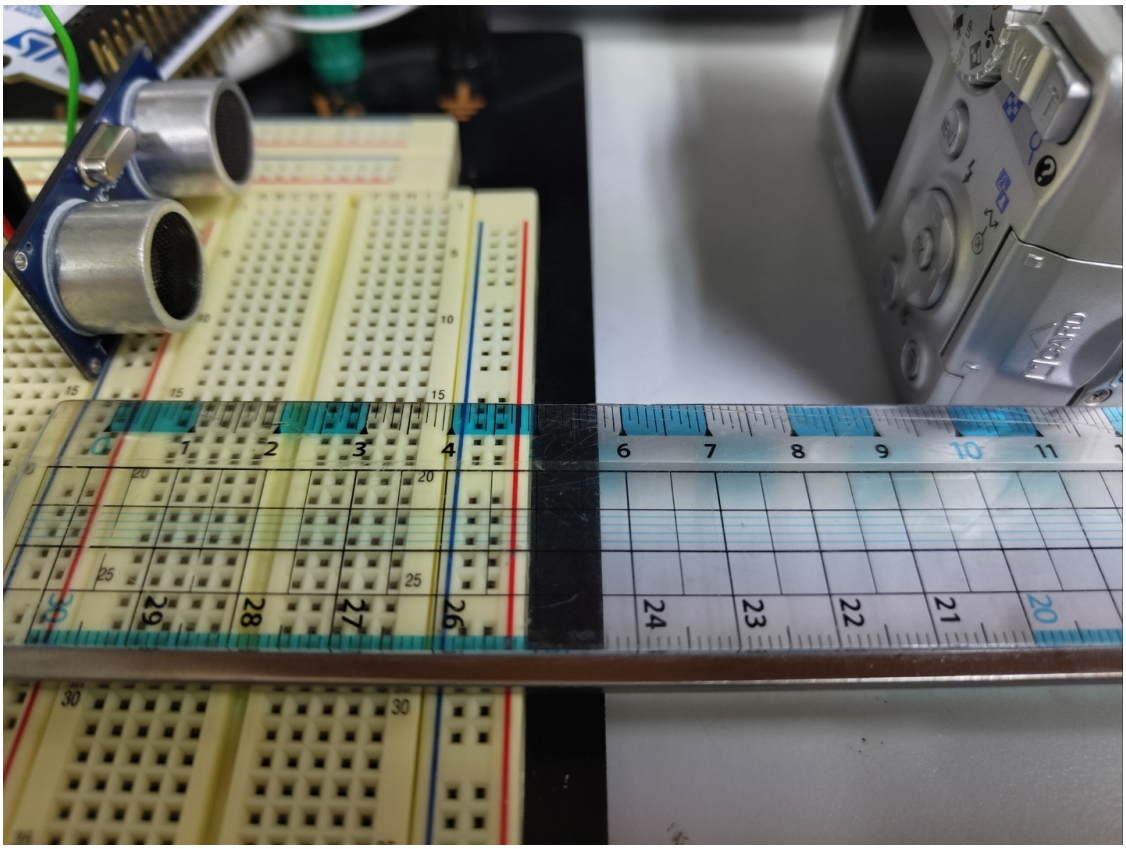
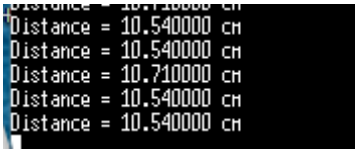
```
#include "stm32f411xe.h"
#include "math.h"
#include "ecSTM32F4v2.h"

uint32_t ovf_cnt = 0;
```

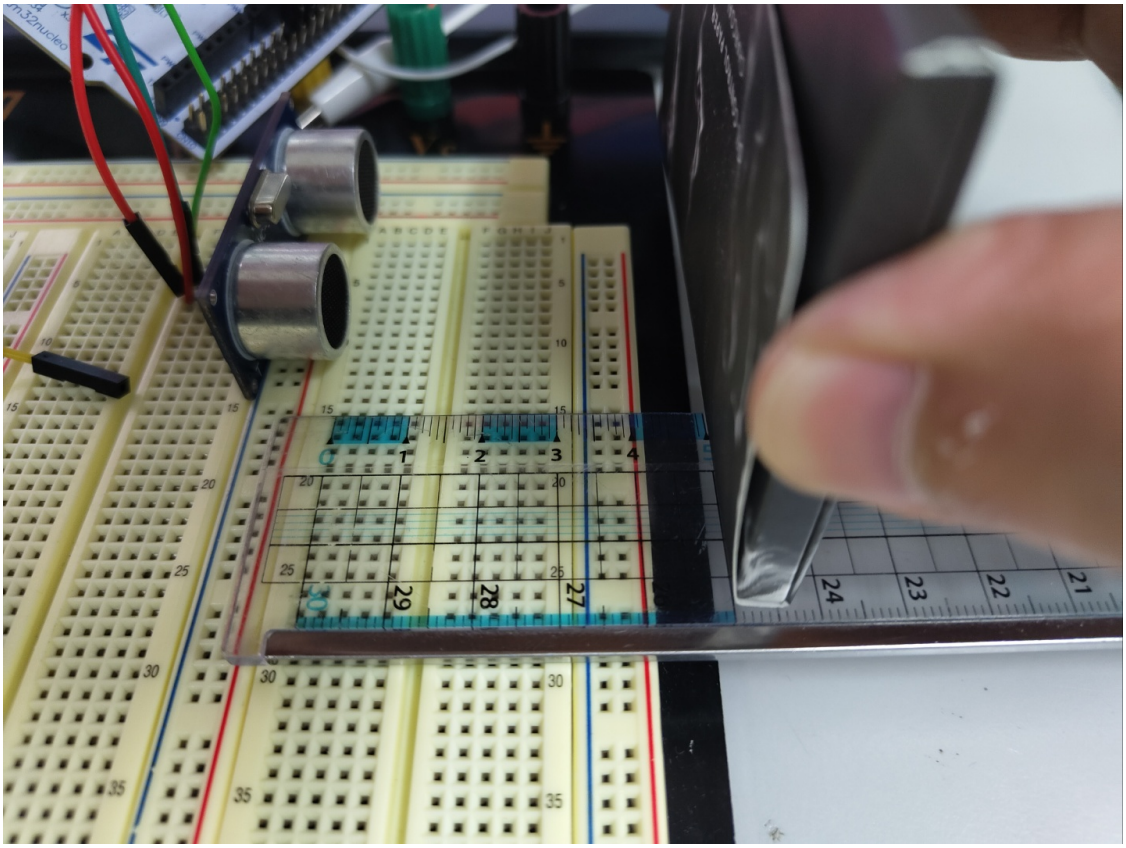
```
float time_interval_us = 0;
float distance = 0;
float timeInterval = 0;
float time1 = 0;
float time2 = 0;

#define TRIG PA_6
#define ECHO PB_6

void setup(void);

int main(void){

    setup();

    while(1){
        distance = (float) timeInterval * 340.0 / 2.0 / 10.0;   // [mm] -> [cm]
        printf("Distance = %f cm\r\n", distance);
        delay_ms(500);
    }
}

void TIM4_IRQHandler(void){
    if(is_UIF(TIM4)){                    // check Update interrupt
        ovf_cnt ++;                                                 // count
        clear_UIF(TIM4);                                // clear
    }
    if(is_CCIF(TIM4, 1)){                               // TIM4_Ch1 (IC1)
Capture Flag. Rising Edge Detect
        time1 = ICAP_capture(TIM4, 1);          // Capture Time_end
        clear_CCIF(TIM4, 1);                // clear
    }
    else if(is_CCIF(TIM4, 2)){                                  // TIM4_Ch2
(IC2) Capture Flag. Falling Edge Detect
        time2 = ICAP_capture(TIM4, 2);                                  //
Capture Time start
        time_interval_us = ((time2 - time1) + ovf_cnt * (TIM4->ARR + 1)) * 10;
//[us]
        timeInterval = time_interval_us / 1000.0;
  //[ms]
        ovf_cnt = 0;                                            // countreset
        clear_CCIF(TIM4,2);                                             //
clear
    }
}


void setup(){
    RCC_PLL_init();
    SysTick_init();
    UART2_init();                  //for serial monitor

// PWM configuration ------------------------------------------------------------
---------
    PWM_init(TRIG);                             // PA_6: Ultrasonic trig pulse
    PWM_period_us(TRIG, 50000);    // PWM of 50ms period. Use period_us()
    PWM_pulsewidth_us(TRIG, 10);   // PWM pulse width of 10us
```

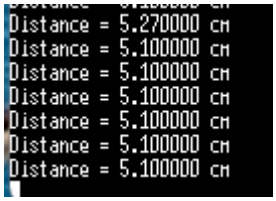
```
// Input Capture configuration ----------------------------------------------------
---------------------
    ICAP_init(ECHO);                  // PB_6 as input caputre
    ICAP_counter_us(ECHO, 10);        // ICAP counter step time as 10us
    ICAP_setup(ECHO, 1, IC_RISE);   // TIM4_CH1 as IC1 , rising edge detect
    ICAP_setup(ECHO, 2, IC_FALL);   // TIM4_CH2 as IC2 , falling edge detect
}
```

## Results

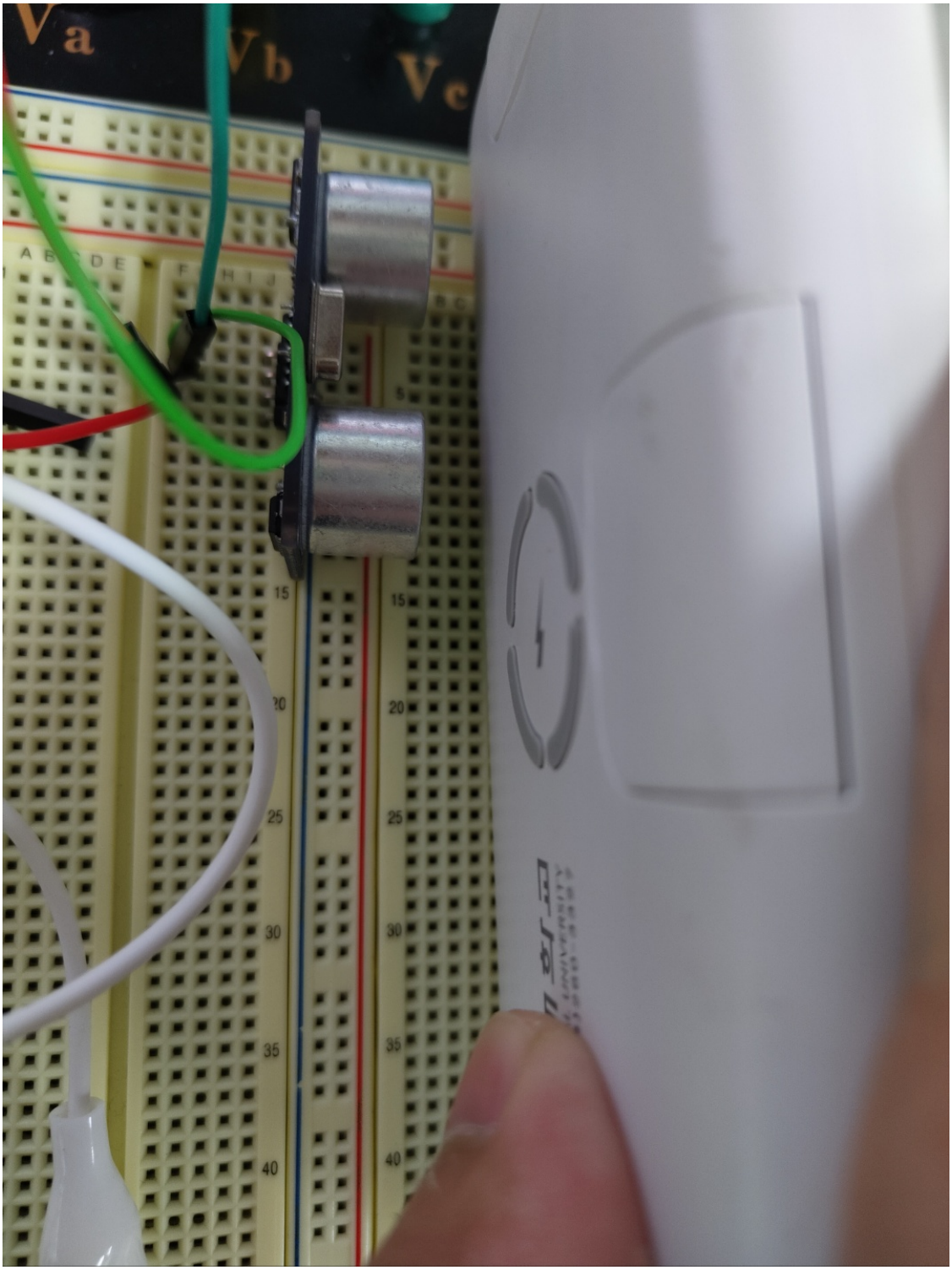
Experiment images and results

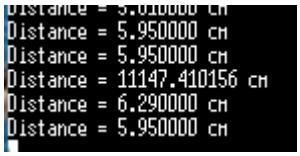
> 1. Display measured distance in 100[mm]
>
> 
>
> 
> 
>
> 2. Display measured distance in 50[mm]






3. Display measured distance in 10[mm]






10mm couldn't be detected correctly due to ultrasonic sensor HC-SR04 range is 2cm - 400cm.

[demo video link]

## Reference


1. Yifeng Zhu - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (2018)
2. https://ykkim.gitbook.io/ec/ec-course/lab/lab-input-capture-ultrasonic (Github)

# Troubleshooting

(Option) You can write Troubleshooting section